

MUIOGO — Proposal for an Iterative OSeMOSYS/MUIO ↔ OG-Core Coupling

A bridge-layer specification for a converged resource-macroeconomy model — Revision 4.2
(Philippines pilot • operational bridge tables with exact file/parameter names • residual-based convergence • configurable iteration order)

0. Resolved scoping decisions (decision log)

Decision	Resolution	Consequence in this proposal
Production setup of first release	Multi-industry OG-Core with an explicit energy industry ($M \geq 2$) from release one	Energy price channel runs through the energy industry's productivity path (Recipe B1) from the start.
Capital-market closure	Partially open economy — investment financed by a realistic mix of domestic savings and foreign inflows	Interest-rate feedback (Recipe A2) active but attenuated; two foreign-financing shares and a world rate become calibration tasks (§13.1).
Pilot	Philippines: OG-PHL + the DESA Philippines OSeMOSYS model	OG-PHL ships as a PyPI package (<code>ogphl</code>) with a <code>Calibration</code> class that patches the OG-Core <code>Specifications</code> object — the exact hook the adapter needs. Pilot section: §13.
CLEWs scope (Rev. 3)	Bridge specified commodity-agnostically: energy, food/agriculture, water	Bridge C (§8). Demand-side food/water rows in release one; supply-side agriculture leg in Phase 2. Rice is the flagship Bridge-C commodity for the Philippines.
Iteration order (Rev. 4)	Configurable entry point; default: enter on the energy side using the model's own calibrated baseline demands, after one anchor run of each model	§10.1. Validation includes a start-from-both-ends test.
Convergence philosophy (Rev. 4)	Convergence judged on residuals (raw model disagreement), never on step size; damping is step-size control only; non-convergence and infeasibility are first-class results with structured reports	§10.2–§10.4.

Decision	Resolution	Consequence in this proposal
Bridge form (Rev. 4.2)	Bridges restated as operational tables per direction — Read from source model (exact output file/key) → Transformation (equation to code) → Write to destination model (exact input parameter) — with two-part plain-English blocks: <i>what is happening</i> and <i>what to code</i>	§§6–8 restructured; name map in §5.3; cheat sheet (§9) aligned.

1. Objective and rationale

OSeMOSYS is a bottom-up, least-cost optimization model of physical systems: given demand trajectories for its commodities, technology costs, and a discount rate, it computes the cheapest build-out and operation plan, plus investment needs, costs, and emissions. Its vocabulary is energetic ("fuel", "technology") but its sets are deliberately generic — a "fuel" is any commodity and a "technology" is anything that transforms one commodity into another — which is what UN DESA's CLEWs models exploit: their commodities include crops, livestock products, water flows, and land, alongside electricity and fuels. OG-Core (and its Philippine calibration, OG-PHL) is a top-down overlapping-generations model of the whole economy: given productivity paths, demographics, and fiscal policy, it computes GDP, consumption, investment, wages, interest rates, industry prices, government revenue and debt — including who, by age and income group, bears which costs.

Each model assumes what the other computes. OSeMOSYS assumes demand paths and a discount rate — macroeconomic outcomes. OG-Core assumes productivity and price formation in energy and agriculture — outcomes of technology choice and resource limits. MUIOGO closes the loop: the models run alternately, exchanging a small set of numbers through a bridge layer, until the exchanged numbers stop changing — a "fixed point" where the resource plan and the macro trajectory are mutually consistent. This two-way soft link with iteration is the established architecture (MESSAGE-MACRO; TIMES-MACRO; the EMEC/TIMES-Sweden link). MUIOGO would be the first such link with a full OLG model on the macro side — adding fiscal and distributional detail no CGE link provides — and the first spanning the full CLEWs commodity space.

Implementation home: MUIOGO inherits MUIO's architecture (Flask backend (API/), frontend (WebAPP/), data in (WebAPP/DataStorage/)); OG-Core/OG-PHL are pure Python. A Python orchestration package inside the MUIOGO backend drives both models and owns every transformation.

2. The two models, reduced to their coupling-relevant interfaces

2.1 OSeMOSYS (as implemented in MUIO) — indexed by region *r*, commodity *f*, technology

t, emission e, year y

Bridge-relevant **inputs**: $\text{SpecifiedAnnualDemand}[r, f, y] + \text{SpecifiedDemandProfile}$ (profiled commodities, e.g., electricity); $\text{AccumulatedAnnualDemand}[r, f, y]$ (unprofiled commodities — crops, water, most fuels); $\text{DiscountRate}[r]$ (one number per region); CapitalCost , FixedCost , VariableCost , OperationalLife ; $\text{EmissionsPenalty}[r, e, y] / \text{AnnualEmissionLimit}[r, e, y]$; $\text{EmissionActivityRatio}$; CapacityFactor , $\text{TotalAnnualMaxCapacity}$ (carriers of climate-scenario and land-limit content in CLEWs).

Bridge-relevant **outputs**: CapitalInvestment , $\text{AnnualFixedOperatingCost}$, $\text{AnnualVariableOperatingCost}$, $\text{AnnualTechnologyEmissionsPenalty}$, NewCapacity , $\text{ProductionByTechnologyAnnual}$, AnnualEmissions , TradeAnnual ; and the solver's **duals** (shadow prices) on each commodity's annual balance — the marginal cost of supplying one more unit — readable from the solver's solution file.

2.2 OG-Core / OG-PHL

M industries, each with output $Y_m = Z_m \cdot F(\text{capital}, \text{public capital}, \text{labor}) \cdot Z[y, m]$ is a per-industry, per-year **productivity dial**; industry M is the reference good and sole producer of investment goods; industries 1...M-1 supply the I household consumption goods through a fixed mapping matrix (io_matrix) — OG-PHL's calibration already ships an output-consumption bridge module for exactly this). Households have minimum required consumption per good ($\text{c_min}, i$) and face per-good, per-year consumption tax rates (tau_c). Government spends $\alpha_G \cdot Y$, transfers $\alpha_T \cdot Y$, invests in public infrastructure $\alpha_I \cdot Y$ (allocated across industries), follows a debt rule. Remittances are modeled explicitly (calibrated at 8.3 % of GDP in OG-PHL — macro-significant for the Philippines). Openness: a share of marginal private capital supplied from abroad at the world rate, and a share of new government debt bought by foreigners.

Bridge-relevant **inputs**: the Z array, tau_c , α_I (+ industry allocation), io_matrix , c_min , consumption shares, openness shares, world rate. Bridge-relevant **outputs** (steady state + transition path): GDP, industry outputs, consumption (total and per good), wages, interest rates, industry prices, revenue/debt. All solved in "shrunk" units (§5.2); exact file locations and key names in §5.3.

2.3 Structural mismatches the bridge absorbs

(1) **No intermediate inputs in OG-Core** — firms use only capital and labor, so firms' energy/water costs must enter as productivity adjustments; households' energy and food enter explicitly as consumption goods. (2) **One discount-rate slot in OSeMOSYS** vs a full interest-rate path in OG-Core — compress to a compound average (Phase 3: year-indexed rate via a code extension). (3) **Different horizons and units** — 25-45 calendar years in physical units and pesos vs ≈ 320 model years in shrunk relative units; the bridge owns windowing, the unit registry, and all conversions. (4) **No land factor in OG-Core** — land scarcity rents reach the macro side only inside the agriculture industry's supply cost (flagged limitation, §14). (5) **Attenuated interest-rate channel** under partial openness — realistic, and stabilizing for the loop.

3. Coupling architecture

The MUIOGO orchestrator owns the configuration, the units registry, and the convergence controller, and sits between two adapters. Each iteration it moves two payload files: `macro_to_energy.csv` (energy, food, and water demands; the discount rate; the carbon price) and `energy_to_macro.csv` (cost indexes per supply bundle; investment shares; emissions; duals).

Component	Responsibilities
Orchestrator (<code>muiogo</code>)	Load config; sequence the loop; apply all recipe transformations; measure residuals; apply damping; write per-iteration audit folders; decide converged / continue / stop-and-report.
OSeMOSYS/CLEWs adapter	Read the MUIO case's data store; write the patched parameters; invoke MUIO's existing run pipeline (GLPK/CBC/HiGHS); parse result CSVs and the solver solution file .
OG-Core/OG-PHL adapter	Instantiate <code>Specifications</code> ; apply <code>ogphl.Calibration</code> ; patch <code>Z</code> , <code>tau_c</code> , <code>alpha_I</code> , openness parameters; run steady state + transition path; extract output paths.

Scoping principle that keeps the bridge small: **only the system boundary crosses it**. Irrigation water, land allocation among crops, fertilizer, energy-for-water — all endogenous inside the LP; the CLEWs nexus stays inside OSeMOSYS. The bridge touches only final demands, supply costs of household-facing bundles, investment totals, emissions, and the discount rate.

4. One-time setup (before any iteration)

S1. Align calendars. Pick base year y_0 = first OSeMOSYS year = OG-Core's period 0; one model period = one year. Exchange window W = OSeMOSYS years $\{y_0 \dots y_N\}$; extend the OSeMOSYS horizon 5-10 years past the reporting window (salvage values handle the tail).

S2. Anchor runs. Run OG-PHL once standalone (no coupling) → the *reference macro paths*. Run OSeMOSYS once standalone on its existing calibrated demands (this solve usually already exists in MUIO) → the *reference cost/price levels*. Every recipe below works in ratios against these anchors, so the bridge steers *trajectories* while base-year *levels* stay exactly as each country team calibrated them.

S3. Money scale. κ = (Philippine GDP in pesos in y_0 , from national accounts) ÷ (OG-Core's unshrunk model GDP in y_0). Used only where absolute pesos are needed (Recipes B3, B4); all price channels use indexes and never need κ .

S4. **Config tables** (filled by economists/energy analysts; read by code): commodity list with drivers and elasticities (B_D); household and firm fuel bundles with expenditure weights (B_P); per-industry energy/water cost shares $s[x,m]$; per-technology public-financing flags; per-commodity damping λ_f ; tolerances; units (PJ, GW, kt, Mm^3 , MP ; pesos of base year on both sides — set in MUIO's unit settings).

S5. **Shared demographics.** Same UN WPP vintage on both sides; the orchestrator checksums the population series. The bridge reads $\text{pop}(y)$ from this shared file, not from either model's output.

5. How to read the recipes

5.1 Glossary (plain English, one line each)

Term	Meaning
Index	A series divided by its start-year value. $1.20 = \text{"20 \% above the start year."}$ Ratios make units and money scales cancel.
Elasticity (η)	The exponent saying how strongly one thing follows another. $1.0 = \text{one-for-one}$; $0.2 = \text{barely responds}$.
Productivity dial ($\text{Z}[y,m]$)	OG-Core knob per industry per year. The industry's product price moves exactly <i>opposite</i> to it: half the dial $\rightarrow$ double the price.
Relative price	OG-Core quotes prices relative to a reference good, not in pesos. That is why price channels only ever match <i>indexes</i> .
Shrunk units	OG-Core divides quantities by population $\times$ a steady growth trend so numbers stay small over 300 years. LEVEL (§5.2) multiplies the trend and population back in.
κ (money scale)	One constant converting model money to pesos, set once so model GDP = actual GDP in the start year.
Discount rate	The single interest rate OSeMOSYS uses to compare money today with money later.
tau_c	A sales-tax-style rate per consumption good per year — our carbon-revenue lever.
alpha_I	The dial for public infrastructure spending as a share of GDP.
Residual	The measured disagreement between the two models on the same quantity, in log terms ($\approx$ percentage gap).

Term	Meaning
Damping (λ)	The fraction of the disagreement corrected per loop. A step size — not a thumb on the scale (§10.2).
Fixed point	A set of exchanged numbers each model hands back unchanged. The converged answer.
Dual / shadow price	The LP solver's report of what one more unit of a commodity would cost — its marginal supply cost.

5.2 Helper functions (implement once, reuse everywhere)

Symbols available to every recipe:

Symbol	Meaning
$\text{pop}(y)$	Population in year y , from the shared UN WPP file (§4-S5)
g	OG-Core's per-year technology growth rate (its parameter g_y)
y_0, W	Base year; the exchange window $W = [y_0 \dots y_N]$
$X(y)$	Any OG-Core "shrunk" output series (GDP, industry output, consumption...)
$\hat{w}(y)$	The OG-Core wage series, in its shrunk form
κ	The money scale from §4-S3

The four helpers, defined once in `transforms.py`:

Helper	Definition	What it does
$\text{LEVEL}(X, y)$	$X(y) \times \text{pop}(y) \times e^{(g \cdot (y - y_0))}$	Un-shrinks a quantity back to its real level
$\text{PESOS}(X, y)$	$\kappa \times \text{LEVEL}(X, y)$	Turns a model quantity into pesos
$\text{IDX}(s, y)$	$s(y) \div s(y_0)$	Turns any series into an index (= 1 in the base year)
$\text{WIDX}(y)$	$\hat{w}(y) \div \hat{w}(y_0)$	Wage-versus-trend index. The shrunk wage ratio is <i>already</i> detrended — use it as-is

5.3 Where things live (name map — declared once, used by every table)

Reading OG-PHL outputs. A run writes pickled dictionaries:

(`OUTPUT_BASELINE/TPI/TPI_vars.pkl`) (the year-by-year transition path — the bridge's source) and (`OUTPUT_BASELINE/SS/SS_vars.pkl`) (steady state). Keys the bridge uses from (`TPI_vars.pkl`):

Key	Content
<code>'Y'</code>	GDP path (shrunk), one value per model year
<code>'Y_vec'</code>	Industry outputs (shrunk), years × industries
<code>'C'</code>	Total household consumption (shrunk)
<code>'r'</code> , <code>'r_p'</code>	Interest-rate paths: firms' rate; household portfolio rate
<code>'w'</code>	Wage path (shrunk form — see WIDX)
<code>'p_m'</code>	Industry price paths, years × industries

Version note: exact key spellings are taken from the pinned (`ogcore`) release; the adapter asserts their presence at startup, and the channel-off equivalence test (§12) catches any drift.

Writing OG-PHL inputs. Attributes of the (`Specifications`) object (`p`), set through (`p.update_specifications({...})`) after (`ogphl.Calibration`) has been applied:

Attribute	Content
<code>p.Z</code>	Productivity dials, years × industries
<code>p.tau_c</code>	Consumption-tax rates, years × consumption goods
<code>p.alpha_I</code>	Public-infrastructure investment as a share of GDP, per year (+ its industry allocation parameter)
<code>p.io_matrix</code> , <code>p.alpha_c</code> , <code>p.zeta_K</code> , <code>p.zeta_D</code> , <code>p.world_int_rate</code>	Fixed at setup (industry↔good mapping; consumption shares; openness; world rate) — not touched inside the loop

Reading OSeMOSYS outputs. The MUIO run pipeline writes one CSV per result variable in the case's results folder, long format (REGION, TECHNOLOGY/FUEL/EMISSION, YEAR, VALUE): (`CapitalInvestment.csv`), (`NewCapacity.csv`), (`AnnualFixed0peratingCost.csv`), (`AnnualVariable0peratingCost.csv`), (`AnnualTechnologyEmissionsPenalty.csv`), (`ProductionByTechnologyAnnual.csv`), (`AnnualEmissions.csv`), (`TradeAnnual.csv`). One derived series the bridge computes itself: **ProductionAnnual(f,y) = Σ over t of ProductionByTechnologyAnnual(t,f,y)**. Duals come from the solver's solution file (keep solution output enabled): the annual commodity-balance rows

((EBb4_EnergyBalanceEachYear...), one per fuel-year — the marginal supply cost) and the emission-cap rows ((E8_AnnualEmissionsLimit...)).

Writing OSeMOSYS inputs. Parameters in the MUIO case's data store, which MUIO serializes into the model datafile at run time: (SpecifiedAnnualDemand), (AccumulatedAnnualDemand), (DiscountRate), (FixedCost), (VariableCost), (EmissionsPenalty), (AnnualEmissionLimit). Input parameters the bridge *reads* from the same store: (CapitalCost), (OperationalLife), (EmissionActivityRatio), and the base-year demands $D(f, y_0)$.

Shared files. The WPP population series; the scenario carbon-price file $\tau_{CO2}(y)$; (bridge_config.yaml) (η_f, v_f, a_f , bundle weights s_f , cost shares $s[x, m]$, θ_L , public_share flags, λ_f, κ , tolerances).

Recipe format in §§6–7: a master table per direction (**Read** — exact source · **Transformation** — the equation to code · **Write** — exact destination), then per-recipe plain-English blocks in two parts: *What is happening* (the substance) and *What to code* (a literal walk-through using the names above).

6. Bridge A — from OG-Core to OSeMOSYS (runs at the start of every iteration)

#	Read from OG-Core	Transformation (code this)	Write to OSeMOSYS
A1	(TPI_vars.pkl) → ('Y') (or one column of ('Y_vec'), or ('C'), per the (B_D) config row); pop(y); config η_f, v_f, a_f ; base demand $D(f, y_0)$ from the MUIO case	$\text{drv_pc}(y) =$ $\text{LEVEL}(\text{driver}, y) \div$ $\text{pop}(y); D_{\text{new}}(f, y) =$ $D(f, y_0) \times$ $\text{IDX}(\text{drv_pc}, y)^{\eta_f}$ $\times \text{IDX}(\text{pop}, y)^{v_f} \times$ $(1 - a_f)^{(y - y_0)}$	(SpecifiedAnnualDemand[r, f, y]) (profiled) or (AccumulatedAnnualDemand[r, f, y]) (unprofiled)
A2	(TPI_vars.pkl) → ('r') (or ('r_p'), per config), years in W	$d = [\Pi \text{ over } y \text{ in } W \text{ of}$ $(1 + r(y))]^{(1/ W)} -$ 1	(DiscountRate[r])
A3 (off by default)	(TPI_vars.pkl) → ('w'); config $\theta_L(t)$	$\text{esc}(t, y) = \theta_L(t) \times$ $\text{WIDX}(y) + (1 -$ $\theta_L(t));$ multiply the baseline cost tables by $\text{esc}(t, y)$	(FixedCost[r, t, y]), (VariableCost[r, t, m, y])
A4	Scenario file → $\tau_{CO2}(y)$ in $\mathbb{P}/tCO_2$	unit conversion only: value = $\tau_{CO2}(y) \times u$ (u	(EmissionsPenalty[r, 'CO2', y]) (cap scenarios: write (AnnualEmissionLimit) instead; see B5)

#	Read from OG-Core	Transformation (code this)	Write to OSeMOSYS
		from the units registry; $\mathbb{P}/t \rightarrow M\mathbb{P}/kt$ $\Rightarrow u = 10^{-3}$)	
A5	Shared WPP file	checksum equality across both models	— (consistency gate)
A6 (Phase 2)	<code>TPI_vars.pkl</code> → the energy/food good's consumption series	Recipe A1 with that series as the driver	same demand parameters

A1 — Demand trajectories (all commodities: energy services, crops, water)

What is happening. OSeMOSYS needs to be told how much electricity, diesel, rice, and water the country will demand each year. Those demands depend on how many people there are and how rich they are — which is exactly what OG-Core computes. So each year's demand is rebuilt from the calibrated start-year demand: scaled up by income per person (softened by an elasticity — rice barely follows income, transport fuel follows it closely), scaled by population, and shaved slightly each year for autonomous efficiency gains. Only *ratios* of OG-Core numbers are used, so its odd internal units cancel out by construction.

What to code. Open `TPI_vars.pkl` from the latest OG-PHL run. For each commodity row in `B_D`: read the named driver array (`'Y'`), a column of `'Y_vec'`, or `'C'`); un-shrink it with `LEVEL`; divide by `pop(y)`; turn it into an index with `IDX` and raise it to the power η_f . Multiply by `IDX(pop, y)v_f` and by `(1 - a_f)(y - y0)`. Multiply the result by the base-year demand `D(f, y0)` already stored in the MUIO case. Write the resulting series into `SpecifiedAnnualDemand` (profiled commodities) or `AccumulatedAnnualDemand` (unprofiled), as flagged in config. Never overwrite the `y0` value; never touch `SpecifiedDemandProfile`.

Notes. Default elasticity library (config-overridable, country review mandatory): residential electricity η 0.6–0.8; industrial energy $\eta \approx 1.0$ with driver = the using industry's column of `'Y_vec'`; transport η 0.8–1.2; rice and staples η 0.1–0.3 (Engel's law); higher-value foods 0.4–0.8; municipal water 0.2–0.4; industrial water 0.6–1.0. $v = 1$ wherever the driver is per-person. `a_f` covers efficiency progress *not already in* OG-Core's growth trend `g` — one config table states which side owns which trend. Irrigation water gets **no row** (the LP derives it from crop choices); export crops are driven by exogenous world-market assumptions, flagged as such in `B_D`.

A2 — Discount rate

What is happening. OSeMOSYS compares money spent today with money spent in 2045 using one interest rate; OG-Core knows what that rate actually is, year by year, given the fiscal scenario. This is the crowding-out channel: fiscal pressure raises interest rates, OSeMOSYS discounts the future more heavily, and its least-cost plan tilts away from build-now-pay-once

options (solar, hydro, irrigation systems) toward pay-as-you-go ones. Under the partially open closure the effect is real but moderate.

What to code. Open `(TPI_vars.pkl)`; read `('r')` (or `('r_p')` if the config says so) for the years in W. Compute the compound average $d = [\prod (1 + r(y))]^{(1/|W|)} - 1$. Write d into the MUIO case parameter `(DiscountRate)` for the model's region. One number in, one number out.

A3 — Wage escalation of running costs (ships OFF by default)

What is happening. Part of every technology's running cost is wages. If the economy's wages run above their long-run trend, operating costs should too — by the wage share of those costs.

What to code. Open `(TPI_vars.pkl)`; read `('w')`. For each year, compute $WIDX(y) = w(y) \div w(y_0)$ — no detrending needed, the shrunk wage ratio is already trend-relative. For each technology, read $\theta_L(t)$ from config and compute $esc(t, y) = \theta_L(t) \times WIDX(y) + (1 - \theta_L(t))$. Multiply the baseline `(FixedCost[t,y])` and `(VariableCost[t,m,y])` values in the MUIO case by $esc(t, y)$ and write them back.

A4 — Carbon price in

What is happening. The scenario's carbon price must reach both models identically. On the OSeMOSYS side it is a per-tonne charge on emissions, which the optimizer then weighs when choosing technologies. (Philippines note: no economy-wide carbon tax is in force — this carries a *proposed* trajectory; fuel excises are the closest existing analogue.)

What to code. Read $\tau_CO2(y)$ from the scenario file. Multiply by the unit factor u from the units registry (pesos/tonne → million pesos/kilotonne means $u = 10^{-3}$). Write the result into `(EmissionsPenalty[r, 'CO2', y])` in the MUIO case. For a cap scenario, write the cap into `(AnnualEmissionLimit)` instead and let Recipe B5 recover the implied price.

A5-A6 — Shared demographics; consumption-driven refinement

A5: no computation — assert both models load the same WPP series (checksum); abort with a clear message if not. A6 (Phase 2): rerun Recipe A1 with the driver switched to the energy or food good's own consumption series from `(TPI_vars.pkl)` — a refinement for residential fuels and food once the multi-good calibration is active.

7. Bridge B — from OSeMOSYS to OG-Core (runs at the end of every iteration)

#	Read from OSeMOSYS	Transformation (code this)	Write to OG-Core
B0	<i>marginal mode:</i> duals on the <code>(EBb4_EnergyBalanceEachYear...</code> rows of the solution file. <i>average mode:</i> <code>(NewCapacity.csv)</code> , <code>(AnnualFixedOperatingCost.csv)</code> , <code>(AnnualVariableOperatingCost.csv)</code> ,	the five formulas in the B0 table below, ending in <code>unitcost(f, y)</code>	— (no write; feeds B1 C4, C5)

#	Read from OSeMOSYS	Transformation (code this)	Write to OG-Core
	<code>AnnualTechnologyEmissionsPenalty.csv</code> , <code>ProductionByTechnologyAnnual.csv</code> + input parameters <code>CapitalCost</code> , <code>OperationalLife</code> , <code>DiscountRate</code>		
B1	unitcost(f,y) for the household bundle HH (weights s_f from <code>B_P</code>); <code>TPI_vars.pkl</code> → <code>'p_m'</code> column of the energy industry; current <code>p.Z</code> column E	$c_E(y) = \sum \text{over } f \text{ in HH of } s_f \times \text{IDX}(\text{unitcost}_f, y); \text{pidx}(y) = \text{IDX}(p_m[:,E], y);$ $Z_{\text{new}}(E,y) = Z_{\text{old}}(E,y) \times \text{pidx}(y) \div c_E(y)$ for y in W; hold the y_N adjustment ratio flat beyond y_N	<code>p.Z</code> column E (via <code>update_specific</code>)
B2	unitcost indexes of the <i>firm</i> bundles $c_x(y)$ (B0 on the firm fuel/water bundles); config $s[x,m]$	$Z_{\text{new}}(m,y) = Z_{\text{baseline}}(m,y) \times \Pi \text{ over } x \text{ of } c_x(y)^{(-s[x,m])}$, for every non-energy industry m	<code>p.Z</code> columns m ≠ E
B3	<code>CapitalInvestment.csv</code> (sum over the coupled technologies); <code>TPI_vars.pkl</code> → <code>'Y'</code> ; κ ; config <code>public_share(t)</code>	$\text{GDP}\mathbb{P}(y) = \text{PESOS}(Y, y); \iota_{\text{pub}}(y) = \sum \text{over } t \text{ of } \text{public_share}(t) \times \text{CapitalInvestment}(t,y) \div \text{GDP}\mathbb{P}(y);$ $\alpha_{I_new}(y) = \alpha_{I_base}(y) + \iota_{\text{pub}}(y)$	<code>p.alpha_I</code> (+ indi allocation toward the energy/water industry public capital); total the private remainder
B4	$\tau_{\text{CO2}}(y)$; the demand values $D(f,y)$ just written by A1 for f in HH; $ef(f)$ from the case's <code>EmissionActivityRatio</code> ; bundle price level $p_{\text{hh}}(y) = \sum s_f \times \text{unitcost}(f,y)$	$E_{\text{hh}}(y) = \sum \text{over } f \text{ in HH of } ef(f) \times D(f,y);$ $\tau_{c_E}(y) = \tau_{\text{CO2}}(y) \times [E_{\text{hh}}(y) \div \sum D(f,y)] \div p_{\text{hh}}(y)$	the energy good's co <code>p.tau_c</code> (held at its value beyond W)
B5	dual on the <code>E8_AnnualEmissionsLimit...</code> rows of the solution file	unit-convert; feed the result into B4 as $\tau_{\text{CO2}}(y)$	<code>p.tau_c</code> (+ report implied carbon price

#	Read from OSeMOSYS	Transformation (code this)	Write to OG-Core
B6 (Phase 3)	<code>TradeAnnual.csv</code> × border prices	net import bill = $\sum$ over f of $\text{borderprice}(f,y) \times \text{imports}(f,y) - \text{the export mirror; enters as a remittance-with-a-minus-sign}$	OG-Core resource cc (code extension down remittance pipe)
B7	<code>AnnualEmissions.csv</code> , <code>NewCapacity.csv</code> , <code>ProductionByTechnologyAnnual.csv</code>	none	dashboard only (ND panels) — not model

B0 — Cost-per-unit extraction (shared sub-recipe)

What is happening. Every price channel needs "what does one unit of commodity f cost in year y." Two ways to answer: *marginal* — ask the solver what one extra unit would have cost (the dual); *average* — rebuild the full annual bill of supplying f and divide by units delivered. The bill has a mortgage-like part (CRF converts each plant's one-off construction cost into equal annual payments over its life, for every plant still alive) plus running costs and emission charges. Philippines default: `average` for the household electricity bundle (retail tariffs are regulated cost-recovery), `marginal` available for sensitivity given the wholesale spot market.

What to code (average mode). Read $d = \text{DiscountRate}$ and, per technology, $L_t = \text{OperationalLife}$ from the case's input data; read `NewCapacity.csv` and `CapitalCost`; read the three cost CSVs and `ProductionByTechnologyAnnual.csv`. Then compute, in order:

Quantity	Formula
$\text{CRF}(d, L)$	$d \times (1+d)^L \div [(1+d)^L - 1]$
$\text{ann_cap}(t, y)$	$\text{CRF}(d, L_t) \times \sum \text{ over build-years } v \text{ in } (y-L_t, y) \text{ of } \text{CapitalCost}(t,v) \times \text{NewCapacity}(t,v)$
$\text{share}(t, f, y)$	$\text{ProductionByTechnologyAnnual}(t,f,y) \div \sum \text{ over } f' \text{ of } \text{ProductionByTechnologyAnnual}(t,f',y)$
$\text{cost}(f, y)$	$\sum \text{ over } t \text{ in } T(f) \text{ of } [\text{ann_cap}(t,y) + \text{AnnualFixedOperatingCost}(t,y) + \text{AnnualVariableOperatingCost}(t,y) + \text{AnnualTechnologyEmissionsPenalty}(t,y)] \times \text{share}(t,f,y)$
$\text{unitcost}(f, y)$	$\text{cost}(f, y) \div \text{ProductionAnnual}(f, y)$ — e.g., M\$ per PJ

$T(f)$ is the config-declared set of technologies supplying f; ProductionAnnual is the derived sum from §5.3. **What to code (marginal mode):** parse the solution file; for each fuel-year,

$\text{unitcost}(f, y)$ = the dual on its `(EBb4_EnergyBalanceEachYear...)` row. *Note:* pre-model legacy plants are treated as sunk; because everything downstream uses the index $\text{unitcost}(f, y) \div \text{unitcost}(f, y_0)$, that omission largely cancels.

B1 — Supply cost → the energy industry's productivity dial

What is happening. In OG-Core, each industry's price moves exactly opposite to its productivity dial: half the dial, double the price. OSeMOSYS has just computed how the true cost of delivering household energy moves over time. So the dial of OG-Core's energy industry is nudged, year by year, until OG-Core's energy price *trajectory* reproduces OSeMOSYS's cost trajectory. The start year is never touched — levels belong to the original calibrations; the bridge steers only how prices move from there. Because both sides are indexes, megapesos-per-petajoule never has to be converted into OG-Core's internal money at all.

What to code. From B0, take $\text{unitcost}(f, y)$ for every fuel f in the household bundle HH; combine into one index $c_E(y) = \sum s_f \times \text{IDX}(\text{unitcost}_f, y)$ using the `(B_P)` weights ($c_E(y_0) = 1$ by construction). Open `(TPI_vars.pkl)`; read the energy industry's column of `('p_m')`; form $\text{pidx}(y) = \text{IDX}(p_m[:, E], y)$. Read the current energy column of `(p.Z)`. For each y in W , set $Z_new(E, y) = Z_old(E, y) \times \text{pidx}(y) \div c_E(y)$. For years beyond y_N , hold the year- y_N ratio of Z_new to the baseline Z flat out to the steady state. Write the column back through `(p.update_specifications({'Z': ...}))`.

B2 — Firms' energy/water costs → productivity wedges on the other industries

What is happening. OG-Core pretends businesses buy no energy or water — only machines and workers. In reality, if a factory spends 5 % of its costs on energy and energy gets 20 % pricier, the factory is effectively 1 % worse off (5 % × 20 %). The same logic applies to water. So each non-energy industry's dial is nudged down by its cost share times the cost increase. First-order approximation, standard in this kind of link, documented as such.

What to code. Run B0 on the *firm-delivered* bundles declared in config (e.g., industrial electricity + diesel; bulk water) to get cost indexes $c_energy(y)$, $c_water(y)$. Read the table $s[x, m]$ from config (filled from PSA input-output tables — economists fill it, code reads it). For every non-energy industry m and year y , set $Z_new(m, y) = Z_baseline(m, y) \times c_energy(y)^{-s[\text{energy}, m]} \times c_water(y)^{-s[\text{water}, m]}$. Write the columns back into `(p.Z)`.

B3 — Investment bill → public infrastructure dial (+ diagnostics)

What is happening. OSeMOSYS reports how many pesos of plants, lines, dams, and irrigation systems get built each year. The publicly financed slice must show up in OG-Core's government budget and debt path — that is what the public-infrastructure dial does. The privately financed slice is deliberately *not* forced into OG-Core: private investment there is decided by households' savings, and the financing-pressure feedback already travels through the interest rate (Recipe A2); forcing both the price and the quantity would over-determine the system. Philippines flags: post-EPIRA generation is largely private → flag ≈ 0 for most power plants; national irrigation, flood control, and dams are public → flag = 1; transmission/distribution per the country team's reading of the concession structure.

What to code. Read `CapitalInvestment.csv`; sum over the coupled technologies per year. Open `TPI_vars.pkl`; read `'Y'`; compute $GDP^P(y) = PESOS(Y, y)$. Compute $\iota_{pub}(y) = \sum_t \text{public_share}(t) \times \text{CapitalInvestment}(t, y) \div GDP^P(y)$, with `public_share(t)` from config. Set $\alpha_{I_new}(y) = \alpha_{I_base}(y) + \iota_{pub}(y)$ and write it into `p.alpha_I`, with the allocation parameter pointing the addition at the energy (or water) industry's public capital. Log the total $\iota(y)$ and the private remainder, and report the private remainder against OG-Core's own implied energy-industry investment as a health check.

B4 — Carbon revenue consistency → sales-tax rate on the energy good

What is happening. A carbon price makes household energy cost more per peso in proportion to how dirty the fuel bundle is. OG-Core's natural lever is the sales-tax rate on the energy consumption good: sized correctly, OG-Core's government automatically collects the same carbon revenue the energy model charges — and what to *do* with that revenue (transfers, labor-tax cuts, debt paydown) becomes an OG-Core scenario lever, which is where the OLG model's distributional payoff lives. Firms' emissions are **not** taxed again here — their carbon cost is already inside the supply costs of B1/B2.

What to code. Read $\tau_{CO2}(y)$ from the scenario file. Take the household deliveries $D(f, y)$ for f in HH — these are exactly the values Recipe A1 just wrote. Read each fuel's emission factor $ef(f)$ from the case's `EmissionActivityRatio` (config maps which delivery technology's factor applies). From B0, form the bundle price *level* $p_{hh}(y) = \sum s_f \times \text{unitcost}(f, y)$. Compute $E_{hh}(y) = \sum ef(f) \times D(f, y)$, then $\tau_{c_E}(y) = \tau_{CO2}(y) \times [E_{hh}(y) \div \sum D(f, y)] \div p_{hh}(y)$. Units check: $(\text{₱}/t \times t/PJ) \div (\text{₱}/PJ) \rightarrow$ a pure rate; 0.04 means 4 %. Write it into the energy good's column of `p.tau_c`, holding the last value beyond W.

B5 — Cap scenarios: shadow price → implied carbon price

What is happening. If the scenario fixes an emissions cap instead of a price, the solver itself reveals what carbon price the cap implies.

What to code. Parse the solution file; read the dual on each year's `E8_AnnualEmissionsLimit...` row; convert units (model-money per emission unit $\rightarrow$ $\text{₱}/tCO_2$). Feed the resulting series into Recipe B4 as $\tau_{CO2}(y)$, and report it on the dashboard as "implied carbon price of the cap."

B6 — Fuel/food import bill (Phase 3)

What is happening. Money leaving the country to pay for imported diesel or rice behaves, in national accounting, like remittances flowing in reverse — and OG-PHL already operates the remittance machinery at 8.3 % of GDP, so the import bill can flow down the same pipe with the opposite sign. Until this lands, the supply-cost channels already carry the import-parity prices embedded in the systems model.

What to code (when scheduled). Read `TradeAnnual.csv`; multiply imports by border prices from config; net off exports; pass the resulting peso series, sign-flipped, into the remittance pathway (an OG-Core-side code extension scoped in Phase 3).

B7 — Reporting (no transformation)

What to code. Copy `AnnualEmissions.csv`, `NewCapacity.csv`, and `ProductionByTechnologyAnnual.csv` into the dashboard data store (NDC/SDG panels). These are displays, not model inputs.

8. Bridge C — the food, water, and land legs (same recipes, new rows)

#	Direction	Exchange	Implementation	Philippines notes
C1	OG-Core → OSeMOSYS	Food demand	Recipe A1 rows for crops/livestock; driver = food-good consumption per person; Engel elasticities	Rice: $\eta \approx 0.1-0.3$, $v = 1$ — demand follows mouths, barely follows wallets
C2	OG-Core → OSeMOSYS	Municipal water demand	Recipe A1; $\eta \approx 0.2-0.4$	district-level data via country team
C3	OG-Core → OSeMOSYS	Industrial water demand	Recipe A1; driver = using industry's column of <code>'Y_vec'</code>	—
C4	OSeMOSYS → OG-Core	Food supply cost	Recipes B0+B1 verbatim, writing the agriculture column of <code>(p.Z)</code> , with the food bundle (FIES expenditure weights)	Land and water scarcity premia are <i>already inside</i> the LP's costs and duals — nothing extra to code
C5	OSeMOSYS → OG-Core	Water supply cost	Default: an extra <code>c_water</code> factor in Recipe B2 via <code>s[water, m]</code> ; optional dedicated water industry via B1	Decision §13.2
C6	OSeMOSYS → OG-Core	Agri/water public investment	Recipe B3 with irrigation, dams, flood control flagged public	National irrigation is public; <code>public_share = 1</code>
C7	OSeMOSYS → OG-Core	Land-use (AFOLU) emissions	Include tagged emission rows in B4's sums and B7's reports	—
C8	(emergent)	Climate-scenario propagation	Nothing to code — yield, hydrology, and capacity-factor shocks entered in CLEWs move supply costs, which reach the macro side through C4/C5/B1/B2	Typhoon/ENSO ensembles are a headline pilot use case

Two structural notes. *Distributional teeth*: OG-Core households carry minimum required consumption per good, so a CLEWs-driven food-price path automatically bites hardest on

households near the subsistence floor — the regressivity of food and carbon price shocks falls out of the model with no extra machinery. *Loop strength*: food's budget share in the Philippines is several times energy's, so the food loop has higher gain than the energy loop even with small Engel elasticities — hence per-commodity damping with a lower default for food (§10.2), and a food-leg-only convergence test in validation. *Honest caveat*: until B6 exists, the domestic food-supply-cost channel may overstate price pass-through for import-buffered staples like rice; the pilot documentation must say so.

9. Cheat sheet — the whole bridge on one page

#	Direction	Read	Compute	Write
A1	OG→OSe	<code>TPI_vars</code> : <code>'Y'</code> / <code>'Y_vec'</code> / <code>'C'</code> ; pop file; base demands	demand formula (A1)	<code>SpecifiedAnnualDemand</code> / <code>AccumulatedAnnualDemand</code>
A2	OG→OSe	<code>TPI_vars</code> : <code>'r'</code> or <code>'r_p'</code>	compound average	<code>DiscountRate[r]</code>
A3	OG→OSe	<code>TPI_vars</code> : <code>'w'</code> ; θ_L	wage escalator	<code>FixedCost</code> , <code>VariableCost</code>
A4	scenario→OSe	<code>tau_CO2(y)</code> file	unit factor	<code>EmissionsPenalty</code>
A5	shared	WPP file	checksum	—
A6	OG→OSe	<code>TPI_vars</code> : per-good consumption	A1 variant	demand parameters
B0	OSe (internal)	result CSVs or solution-file duals	<code>unitcost(f,y)</code>	feeds B1/B2/B4/C4/C5
B1	OSe→OG	B0 household bundle; <code>'p_m'</code> energy column	dial update $Z \times p \div c$	<code>p.Z[:,E]</code>
B2	OSe→OG	B0 firm bundles; <code>s[x,m]</code>	wedge $Z \times$ $\Pi c^{(-s)}$	<code>p.Z[:,m≠E]</code>
B3	OSe→OG	<code>CapitalInvestment.csv</code> ; <code>'Y'</code> ; κ ; flags	GDP shares	<code>p.alpha_I</code> (+ allocation); logs
B4	OSe→OG	<code>tau_CO2</code> ; A1 demands; <code>EmissionActivityRatio</code> ; <code>p_hh</code>	ad-valorem rate	<code>p.tau_c[:,E]</code>
B5	OSe→OG	dual on <code>E8_AnnualEmissionsLimit...</code>	implied price → B4	<code>p.tau_c</code> ; report

#	Direction	Read	Compute	Write
B6	OSe→OG	<code>TradeAnnual.csv</code> × border prices	reverse-remittance wedge	resource constraint
B7	OSe→dashboard	emissions/capacity/production CSVs	—	dashboard
C1- C3	OG→OSe	as A1	A1	demand parameters (crops, water)
C4	OSe→OG	B0+B1 on the food bundle	dial update	<code>p.Z[:,A]</code>
C5	OSe→OG	B2 + s[water,m]	wedge	<code>p.Z</code>
C6	OSe→OG	B3, irrigation/dams public	GDP shares	<code>p.alpha_I</code>
C7	OSe→OG	AFOLU emission rows	include in B4/B7	<code>p.tau_c</code> ; dashboard
C8	emergent	—	nothing	—

10. The iteration loop

10.1 Where the loop starts (and why it barely matters)

If the loop converges, the entry point does not change the answer — a fixed point is a fixed point; entry order changes only the path and the iteration count. Structurally, OSeMOSYS cannot run from nothing (it needs demands) while OG-Core's calibration is self-contained — but the MUIO model already *contains* calibrated baseline demands built from official projections, which are themselves an implicit macro story. So the real choice is which baseline anchors iteration 1: the country team's official energy baseline, or OG-PHL's macro baseline. **Default (config `start_with: energy`)**: after the two anchor runs of §4-S2, enter on the energy side reusing the existing calibrated OSeMOSYS solve (typically already cached in MUIO — cost ≈ zero), then let OG-PHL respond. This preserves the validated official baseline as the reference, and it puts the cheap model first so the expensive model always reacts to fresh systems data. `start_with: macro` is the mirror image. Validation (§12) runs both and checks they meet at the same fixed point; if they do not, that is a *finding* — multiple self-consistent futures — reported as such.

10.2 The update rule, and what damping is (and is not)

For each exchanged series, define the **residual** = the raw disagreement, measured *before* any damping:

$\text{res_D}(f, y) = \ln D_{\text{proposed}}(f, y) - \ln D_{\text{used}}(f, y)$ — where "proposed" is what the latest macro run implies via Recipe A1, and "used" is what the last OSeMOSYS run actually consumed.

$\text{res_p}(g, y) = \ln c_g(y) - \ln \text{pidx}_g(y)$, for each supply bundle $g \in \{E, A, (W)\}$ — what OSeMOSYS says the cost does versus what OG-Core's price did.

Update with per-commodity relaxation:

$$x_{\text{next}}(f, y) = x_{\text{used}}(f, y) + \lambda_f \times \text{res_D}(f, y)$$

with defaults $\lambda = 0.5$ for energy and water, $\lambda = 0.3$ for food. The dial updates of B1/C4 carry their own built-in damping through their iterative form.

Plain English: each loop, measure how far apart the two models still are, then move only a *fraction* of the way toward the other model's answer — like adjusting a shower tap gently instead of cranking it, so you do not ping-pong between scalding and freezing. Two facts make this honest. First, damping cannot move or invent a solution: if the models already agree, the residual is zero and the update changes nothing — for any λ ; the damped and undamped procedures have *identical* fixed points. Second, success is declared **only on the residuals** — never on step size. With a tiny λ the steps shrink, but the disagreement gauge stays loud; heavy damping can therefore never fake convergence.

10.3 The loop

1. **Anchor** (§4-S2) and initialize per §10.1.
2. **Systems step**. Apply A1-A4 (and C1-C3); solve the LP; run B0; compute the feedback set.
3. **Macro step**. Apply B1-B5 (and C4-C7); patch `Specifications` via `ogphl.Calibration`; solve steady state + transition path, warm-started from the previous iterate; extract paths.
4. **Measure residuals** (§10.2). **Converged** when $\max |\text{res_D}| < 5 \cdot 10^{-3}$, $\max |\text{res_p}| < 5 \cdot 10^{-3}$, and the year-by-year GDP change is below 10^{-3} , simultaneously.
5. Otherwise **damped update**, safeguards ($\pm 10\%$ per-iteration cap on any demand element; oscillation detector — sign-flipping residuals — auto-halves the offending λ_f), and return to step 2. Hard cap: 15 iterations $\rightarrow$ non-convergence report (§10.4). Optional Anderson acceleration once the basic loop is validated.

Runtime budget: the macro step dominates — a full OG-PHL baseline-plus-reform solve runs 35 minutes to 2+ hours on a typical machine, so a per-iteration warm-started solve is the cost driver and a coupled run is an overnight job in release one; the LP solves in seconds-to-minutes with CBC/HiGHS. Warm-starting OG-Core from the previous iterate is the single most valuable optimization. MESSAGE-MACRO-class experience: 3-10 iterations for policy-relevant scenarios.

10.4 Failure taxonomy — when "no convergence" is the result

Symptom	What it means	System response
LP reports <i>infeasible</i>	The scenario is physically impossible as specified (e.g., the emissions cap cannot be met with the available technologies)	Stop. Report the binding constraints. This is a headline result, not an error.
OG-Core SS/TPI fails to solve or paths explode	The macro side cannot carry the inputs (e.g., the debt rule is unsustainable under the investment bill)	Stop. Report the offending paths. Also a result.
Residuals shrink steadily	Normal convergence	Converged output + convergence log.
Residuals oscillate, but shrink when λ is lowered	Numerical ping-pong , not economics — the artifact damping exists to remove	Auto-halve λ_f ; continue; note in log.
Residuals will not shrink for <i>any</i> λ	The two models genuinely cannot agree under these assumptions — the loop's economic gain exceeds one (suspect first: food elasticities $\times$ food budget share)	Stop at max-iter. Emit the non-convergence report : residual trajectories per commodity, estimated loop gain, the driving elasticity/share. A first-class deliverable — exactly the "this scenario is not coherent" signal.
<code>(start_with: energy)</code> and <code>(start_with: macro)</code> converge to different answers	Multiple self-consistent equilibria	Report both fixed points side by side. A finding.

The philosophy in one line: damping separates noise from signal — it removes the failures that are about the *algorithm* so that the failures that remain are about the *world*, and those are reported, loudly, as results.

11. Software design within MUIOGO

`(muiogo/)` package in the backend: `(config.py)` (typed loader for `(bridge_config.yaml)`: window, κ , units, `(B_D)`, `(B_P)`, elasticity library, cost shares, public flags, λ_f , tolerances, `(start_with)`, channel switches); `(adapters/osemosys.py)` (read MUIO case data, write patched parameters, invoke the existing MUIO run pipeline, parse result CSVs *and the solution file*);

(`adapters/ogcore.py`) (instantiate (`Specifications`), apply (`ogphl.Calibration`), patch (`Z`), (`tau_c`), (`alpha_I`), openness parameters; run SS + TPI; extract paths; assert the §5.3 key map at startup); (`transforms.py`) (Recipes A1-A6, B0-B5, C-rows — each a pure, unit-checked function; nothing hard-codes "energy": commodities are config rows); (`loop.py`) (§10 controller, residual ledger, failure taxonomy); (`reporting.py`) (dashboard data, non-convergence report, NDC/SDG panels).

Per-iteration artifacts in (`WebAPP/DataSource/<model>/coupled_runs/<case>/iter_k/`): (`macro_to_energy.csv`), (`energy_to_macro.csv`) (long format: variable, commodity, year, value, unit), patched parameter files, both models' native outputs, (`manifest.json`) (config hash, versions, residuals). Reproducible and auditable — country-office grade.

UI: a "Coupled run (OG-CLEWS)" tab — pick the OSeMOSYS case and the OG-PHL calibration, toggle channels, set λ_f /tolerances/(`start_with`), launch asynchronously, watch per-commodity residuals per iteration plus GDP/price/demand/emissions trajectories per iterate; results flow into MUIO's pivot tables plus macro and distributional panels. Dependency policy: pin (`ogcore`)/(`ogphl`) releases; vendor MUIO's OSeMOSYS model file; keep all bridge logic out of both upstream codebases.

12. Validation plan

Channel-off equivalence (all switches off $\Rightarrow$ bit-identical standalone MUIO and OG-PHL results). Unit tests per recipe with round-trip unit assertions (PJ, kt, Mm^3 , MP). Synthetic fixed-point problem with a known answer to validate the controller, residual gauge, damping, and stopping rules. Single-leg convergence tests: energy-only, food-only (the high-gain case), water-only. **Start-from-both-ends test** (§10.1). Sensitivity fans over $\lambda_f \in \{0.3, 0.5, 0.8\}$ and $\pm 30\%$ on elasticities and cost shares. Per-iteration accounting cross-checks against Philippine national accounts ranges: energy cost share, food expenditure share, agriculture value-added share, investment shares. Benchmark experiment: a proposed carbon-price trajectory with revenue recycled to transfers, coupled vs uncoupled — expecting modest GDP deviations but materially different energy and food prices, demands, and fiscal/distributional paths, consistent with the soft-linking literature.

13. Roadmap and pilot

Phase 0 — engineering bring-up (internal, ≈ 4 weeks). Orchestrator skeleton, both adapters, exchange schema, residual ledger and controller; synthetic fixed-point validation; end-to-end smoke test (CLEWs demo + out-of-the-box OG-Core). Not a release.

Phase 1 — first working release ($\approx 10\text{--}14$ weeks). Multi-industry OG-PHL with an explicit energy industry; partially open closure; the DESA Philippines OSeMOSYS model. Channels: A1 across *all* demand commodities present in that model (energy services + the C1-C3 food/water rows — nearly free once A1 exists), A2, A4, B0-B5, B7; A3 available but off.

Safeguards, §12 validation suite, documented CLI. Deliverable: a reproducible coupled Philippine baseline plus one carbon-price scenario, including the convergence/non-convergence reporting.

Phase 2 — CLEWs supply legs + product integration. Agriculture/food industry (C4, C6, C7; $M = 3$) with `(io_matrix)`, consumption shares, and `(c_min)` recalibrated from FIES — leaning on OG-PHL's existing output-consumption bridge module; water as wedge by default (C5), industry optional. Flask endpoints, UI tab, async jobs, dashboards, packaging, docs, performance profiling. Second country to prove generality.

Phase 3 — extensions. Year-indexed `(DiscountRate)` in the bundled OSeMOSYS code (unlocking the full $r(y)$ path); B6 trade wedges for fuel **and rice** through the remittance pipe; population-driven land-availability channel; employment satellite accounts; batch climate-ensemble tooling on C8 (typhoon/ENSO scenarios as a flagship Philippine use case).

13.1 Calibration tasks from the closure decision

Foreign share of marginal private-capital supply; foreign share of new government-debt purchases; world real interest-rate path — each from documented Philippine sources (starting from OG-PHL defaults), each with a $\pm$ range in the sensitivity suite. Note the interaction worth documenting: remittances at $\sim 8\%$ of GDP are a large, already-modeled external inflow; the openness shares govern *capital* flows on top of that.

13.2 Remaining decisions

(1) Confirm the DESA Philippines OSeMOSYS model version and its commodity list — this fixes `(B_D)/(B_P)`, the base year, and the technology detail for B3/C6. (2) `(B_D)` contents and the elasticity library, reviewed with country analysts (rice, fish, electricity, transport fuels at minimum). (3) Whether water is a distinct consumption good (industry W) or stays a cost wedge. (4) Horizon: planning vs reporting window and the matching OG-PHL base year. (5) Compute budget: given 35 min–2 h+ per full OG-PHL solve, decide laptop vs workstation expectations — this sets how aggressive Phase-2 performance work must be. (6) Pilot scenario set: a proposed carbon-price trajectory, an accelerated-renewables/coal-transition case, and a climate-ensemble (typhoon/ENSO) case are the natural candidates; the choice determines which channels are exercised hardest in validation.

14. Risks and known limitations

Non-convergence where it is *not* informative is mitigated by per-commodity damping, change caps, and single-leg tests; where it *is* informative it is reported, per §10.4. Double counting of trends between g , a_f , and OSeMOSYS cost assumptions — prevented by the trend-ownership config table. Elasticities drive loop strength — shipped as a documented default library with mandatory country review. The firm-side channel (B2) is first-order only; OG-Core's uniform depreciation cannot represent asset lifetimes (those live in OSeMOSYS). No land factor in OG-Core: land rents reach the macro side inside the agriculture price and accrue to capital owners — adequate to first order, wrong about factor incomes, documented honestly. Import-buffered

staples (rice) overstate domestic price pass-through until B6 lands — stated in pilot outputs. Subsistence/own-consumption agriculture is imperfectly captured by market-price channels. One national economy: a multi-region OSeMOSYS model must designate the Philippines as the coupled region. OG-PHL runtime bounds the practical iteration count; performance is a Phase-2 deliverable, not an afterthought.

15. The bridge in one paragraph

OG-PHL hands OSeMOSYS a macro story — how many people there are and how rich they are (demand levels for electricity, fuels, rice, and water, via population and Engel/income elasticities), how expensive capital is (one compound-average interest rate, partly disciplined by world markets), how expensive labor is, and what the carbon price is. OSeMOSYS hands back a resource story — what energy and food really cost to supply (as cost indexes that retune the productivity dials of the energy and agriculture industries, with land and water scarcity already priced in), what the build-out costs the public budget (an addition to the infrastructure-spending dial), what the carbon instrument collects (a sales-tax-equivalent on household energy), and what is emitted. A gently damped loop runs the two stories against each other, declares agreement only when the measured disagreement itself vanishes, and treats every other ending — physical infeasibility, fiscal unsustainability, irreconcilable assumptions, multiple equilibria — as a named, reported finding. The converged output is a climate-land-energy-water plan the Philippine economy can afford, a macro trajectory whose resource assumptions are physically feasible, and a household-level account of who pays.